Pengembangan Lokal di Windows (WSL2)

Diterjemahkan dari `docs/final/local-setup-wsl2.md` pada 8 November 2026.

- **Status:** final (disetujui 2026-11-08, D-166)
- **Terakhir diperbarui:** 2026-11-08

Cara pengembang menjalankan SIGRAK di laptop Windows sebelum kode masuk ke staging dan produksi. Pengembangan lokal memakai **WSL2 dengan Ubuntu 24.04**, bukan Laragon (D-166). Ubuntu 24.04 sama dengan VPS (D-086), sehingga Horizon, `phpredis`, dan `pcntl` bekerja sama seperti di server.

Versi mengikuti server-setup.md; di sini hanya dicatat perbedaan untuk lokal.

1. WSL2 dan Ubuntu

Di PowerShell sebagai Administrator:

```
wsl --install -d Ubuntu-24.04
```

Mulai ulang Windows, buka "Ubuntu 24.04", lalu buat pengguna Linux.

- Simpan kode **di dalam Linux** (`~/sigrak`), jangan di bawah `/mnt/c/...`. Di `/mnt/c`, Composer, Vite, dan tes berjalan beberapa kali lebih lambat dan pemantauan perubahan file tidak andal.
- Sunting dengan VS Code dan ekstensi **WSL**-nya: jalankan `code .` dari terminal Ubuntu di dalam folder proyek.

2. PHP, MySQL, Redis

```
sudo LC_ALL=C.UTF-8 add-apt-repository -y ppa:ondrej/php && sudo apt update
```

```
sudo apt -y install php8.5-cli php8.5-mysql php8.5-redis php8.5-mbstring php8.5-xml php8.5-curl
php8.5-zip php8.5-gd php8.5-intl php8.5-bcmath mysql-server redis-server unzip git
```

- Tidak perlu PHP-FPM atau Nginx di lokal: `php artisan serve` sudah cukup.
- `mysql-server` dari Ubuntu adalah versi 8.0; produksi dan CI memakai 8.4. Untuk kerja lokal itu tidak masalah; CI menangkap perbedaan versi.
- Redis dari Ubuntu lebih lama daripada versi 8.2 di produksi. Di lokal juga tidak masalah.

Pasang Composer dengan installer resmi dari getcomposer.org.

3. Node.js 24 LTS

Pasang dengan `nvm` (github.com/nvm-sh/nvm), lalu:

```
nvm install 24
```

4. Database

MySQL di Ubuntu hanya mendengarkan di `127.0.0.1`, jadi pengguna lokal dengan `'%'` tidak bisa dijangkau dari luar. `testing` adalah database yang dipakai rangkaian tes Pest (`phpunit.xml`).

```
sudo mysql -e "CREATE DATABASE sigrak; CREATE DATABASE testing; CREATE USER 'sigrak'@'%' IDENTIFIED BY
'secret'; GRANT ALL ON sigrak.* TO 'sigrak'@'%'; GRANT ALL ON testing.* TO 'sigrak'@'%';"
```

5. Gotenberg (PDF)

Pasang **Docker Desktop** di Windows dan aktifkan *Settings → Resources → WSL integration* untuk Ubuntu 24.04. Lalu, di Ubuntu:

```
docker run -d --name gotenberg --restart unless-stopped -p 127.0.0.1:3000:3000
gotenberg/gotenberg:8.39.0-chromium
```

Di lokal, Gotenberg berjalan tanpa basic auth dan tanpa penguncian jaringan dari D-059.

6. `.env`

```
cp .env.example .env
```

Isi setidaknya nilai-nilai berikut. Ketiga hostname memakai `.localhost`: Chrome dan Edge mengarahkan setiap nama `*.localhost` ke mesin sendiri, sehingga file hosts Windows tidak perlu diubah.

```
APP_ENV=local
APP_DEBUG=true
APP_URL=http://app.sigrak.localhost:8000
APP_HOST=app.sigrak.localhost
EXAM_HOST=ujian.sigrak.localhost
ADMIN_HOST=admin.sigrak.localhost

LOG_STACK=single
LOG_LEVEL=debug

DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=sigrak
DB_USERNAME=sigrak
DB_PASSWORD=secret

REDIS_HOST=127.0.0.1
REDIS_PORT=6379

QUEUE_CONNECTION=redis
CACHE_STORE=redis
SESSION_DRIVER=redis
SESSION_SECURE_COOKIE=false

TELESCOPE_ENABLED=true

LARAVEL_PDF_DRIVER=gotenberg
GOTENBERG_URL=http://127.0.0.1:3000
```

Tes membaca `DB_HOST`, `DB_USERNAME`, dan `DB_PASSWORD` dari `.env` dan memakai database `testing`.

7. Menjalankan pertama kali

```
composer install && npm ci
```

```
php artisan key:generate && php artisan migrate --seed
```

8. Kerja sehari-hari

```
composer dev
```

`composer dev` menjalankan `php artisan dev`, yang menyalakan `serve` (port 8000), `queue:listen`, `pail` (log), dan Vite sekaligus. Buka:

- Aplikasi sekolah: `http://app.sigrak.localhost:8000`
- Situs ujian: `http://ujian.sigrak.localhost:8000`
- Admin (Filament, Horizon, Telescope): `http://admin.sigrak.localhost:8000`

`queue:listen` hanya mengerjakan antrean `default`. Untuk mencoba Horizon atau antrean bernama dari `config/horizon.php`, jalankan `php artisan horizon` di terminal kedua.

Sebelum push, jalankan pemeriksaan yang juga dijalankan CI:

```
vendor/bin/pint --test && vendor/bin/phpstan analyse --memory-limit=1G && composer test
```

```
npm run lint && npm run check && npm run build
```

Pemecahan masalah

- **Halaman tampil tanpa CSS/JS:** buka `public/hot`. Jika isinya `[::1]`, jalankan Vite dengan `npm run dev -- --host 127.0.0.1` (Windows tidak selalu meneruskan IPv6 ke WSL).
- **`localhost:8000` tidak terbuka dari Windows:** pastikan penerusan localhost WSL aktif (bawaan; `localhostForwarding=true` di `%UserProfile%\.wslconfig`), lalu jalankan `wsl --shutdown` dan buka lagi Ubuntu.
- **`composer install` atau `npm ci` lambat:** proyek berada di bawah `/mnt/c`. Pindahkan ke `~/`.
- **`docker: command not found` di Ubuntu:** aktifkan WSL integration di Docker Desktop dan buka ulang terminal.